Review changelog: batch "projects" (react-mastery/projects/01 to 11)

Reviewer: technical validation pass, 2026-10-22. Scratch work (all executable evidence) is in `/Users/todorbra/Projects/ReactLearning/tools/sandbox/scratch/projects/` (`p01/src0` ... `p10/src0`, `p11/{broken,fixed,tests}`, `p03b` for React Router 8). Environment: Node 24.16, React 19.3.1, TypeScript 5.9, Vitest 3.2 (and 5.0.5 for P11), jsdom 30, Vite 8.3.5, `@vitejs/plugin-react` 6.1.2, TanStack Query 5.104, TanStack Virtual 3.14, Zod 4, React Router 7.18.4 and 8.5.0, ESLint 10.13, dependency-cruiser 18.7.

Important context correction: **React Router 8 is the current major** (8.5.0 is `latest` on npm; `react-router-dom` stops at 7.18.4). The brief says v7 is current; it is not. Lessons 03 and 10 now say so, and their data-mode code was run on both 7.18.4 and 8.5.0.

Method: each lesson's reference code was extracted from the markdown into a scratch project (missing helper files stubbed), typechecked with `strict` + `noUncheckedIndexedAccess`, and the claims that carry weight were executed with Vitest + RTL (jsdom), node scripts, `vite build`, ESLint and dependency-cruiser runs.

Project 11: broken app (most important)

Extracted the full broken source (§6) and fixed source (§C) verbatim into `p11/broken` and `p11/fixed`.

Check	How	Result
Both sources typecheck (strict, noUncheckedIndexedAccess)	`tsc`	Confirmed (0 errors, both)
Both build with Vite 8; bundle effect of D8	`vite build`	Confirmed. Entry 364 kB (118 kB gz) broken vs 266 kB (83 kB gz) + 0.5 kB Reports chunk fixed. Visualizer `--mode analyze` works on Vite 8
D6: 5,000 `.row` nodes; Reports mounted while hidden (S9)	`tests/broken.test.tsx`	Confirmed; fixed renders 23 rows (660 px viewport)
D2/S2: idle re-renders everything each second	formatAge counter	Confirmed: 5,000 row renders per second (15,000 in 3 s); fixed: 0 row renders per second, only `Clock` (3 in 3 s), rows update once a minute
D5/S3: search race, final list = shortest prefix	fake timers with the real mockApi	Confirmed: input `login`, list shows the 3,662 results for `l`, not the 526 for `login`. Fixed: debounced, exactly 1 request, all rows match

Check	How	Result
D5/S4: even then odd click -> heading odd, body even	same	Confirmed. Fixed: always last selected; first request's signal aborted
D12: stale content and reply draft survive switching	same	Confirmed (old description + "hello A" remain; "Loading..." shown). Fixed: reply cleared
D9/D4/S6: mutation visible late	same	Confirmed and **refined**: the row only updates at the next 1 s tick, the detail `<select>` snaps back to the old value until then, and the **header stats never update at all** (effect keyed on the unchanged array reference). Lesson text only claimed "up to one second". Fixed: immediate, stats update
D3/S5: index keys move the star	same	Confirmed (star stays on row 0 while the ticket changes). Fixed: star follows the ticket id
D7/S7: interval never cleared; re-tick doubles rate	same	Confirmed (untick: Total kept growing; re-tick: +4 in 4 s instead of +2). Fixed: stops, +2 per 4 s
D7 Strict Mode claim "a single tick creates two intervals"	spied `setInterval` under `<StrictMode>`	**Wrong.** Exactly one interval per tick (Strict Mode's extra mount/cleanup/mount happens only on first mount, where the effect returns early). Corrected in text
D7/S8: resize listener leak	spied add/removeEventListener	Confirmed: +1 per open in prod; **+2 per open under Strict Mode** (added to text). Fixed: balanced
D10: comparator invoked ~60,000 times	counted calls	Confirmed (54,974 for 5,000 items)
D4: two commits per filter change	counted provider-level renders	Confirmed (2)
D1/D11	by code and render counts	Confirmed
D8 budget test regex (`moment.version	MomentJS	lodash.com`)
C.2 text "Intl.RelativeTimeFormat yields 'in 0 minutes'"	node	**Wrong**: `numeric:'auto'` gives "this minute". Corrected
C.2 `formatAge` with 1-minute clock	test	**Latent bug in the fix**: a live ticket created after the last minute tick formats as "in 1 minute". Clamped to the past (`Math.min(0, ...)`)

Check	How	Result
Search results are a snapshot	test	**Latent bug (both versions)**: with an active search, a status change updates `tickets` and the detail panel but the row (rendered from stale `results`) keeps the old badge. Documented in §D interactions
jsdom + `@tanstack/react-virtual`	tests	Library 3.14 reads `clientWidth/clientHeight`, not `getBoundingClientRect`; §11 and C.8 now say exactly what to stub (snippet added)
`useVirtualizer` / React Compiler incompatibility	read compiler `DefaultModuleTypeProvider.ts` on GitHub	Confirmed; marker removed
`spyOn` on ESM exports for the F sketches	used in fixed tests	Works on Vitest 3.2 and 5.0.5
package.json versions	built/tested	Updated to Vite ^8, plugin-react ^6, Vitest ^5, jsdom ^30, visualizer ^7, TS ^5.9; added `/// <reference types="vitest/config" />`; fixed tests also pass on Vitest 5.0.5

Project 03: multi-page dashboard (React Router)

Extracted all code; ran the four §9.2 integration tests and extra behavior tests on RR 7.18.4 and 8.5.0.

- **Corrected (significant):** `ErrorBoundary` on the shell route replaces the whole shell, so the lesson's own test "404 keeps the shell" **failed**. Reference `createRoutes` now puts the boundary on a pathless layout route inside the shell (renamed `RouteErrorBoundary`); test then passes.
- **Corrected:** `useLoaderData() as ...` replaced by `useLoaderData<ReturnType<typeof createXLoader>>()`; verified it types the result (including `Date`) on both versions.
- **Verified, markers removed:** `react-router/dom` RouterProvider (7 and 8); interrupted navigation aborts the first loader's signal; parent and child loaders start in parallel; `revalidate()` re-runs a failed route's loader; `router.navigate(-1)`; `lazy` function form; `defer` is gone; `Link prefetch` is framework-mode only (docs); `expectTypeOf` exists.
- **Added (new, found by running):** two test-environment traps. (1) Under jsdom, React Router's `new Request(url, {signal})` throws "Expected signal ... to be an instance of AbortSignal" and every test fails; workaround documented. (2) On RR 7.x, importing `RouterProvider` from `react-router/dom` and hooks from `react-router` can load two module copies under Vitest ("useRouteError must be used within a data router"); using `RouterProvider` from `react-router` in tests fixes it. RR 8 is ESM-only and unaffected.
- Version note rewritten (RR 8 current; Node 22.22+/React 19.2.7+).

Project 09: SSR + streaming

Built the pipeline (`renderToPipeableStream` wrapper, document, cache, serializer) and ran an in-process stream test plus a client `use()` test.

- **Corrected (syntax error):** the serializer's regex and map keys contained *literal* U+2028/U+2029 characters; U+2028 is a line terminator, so `/[<>&<LS><PS>]/g` is an unterminated regex literal (TS1161) and the lesson's code did not compile. Replaced with ` / ` escapes.
- Confirmed: shell flushes with skeleton before reviews; later chunk carries review text + data script, payload escaped (`</script>` neutralized); `<title>` , `<meta>` , `<link precedence>` hoisted into `<head>` (`data-precedence`); bot UA waits for `onAllReady` and output has no skeleton; rejected boundary => 200, `onError` called, React `$RX` client-render marker.
- **Corrected wording:** bot path is not literally "a single flush" (React wrote 2 chunks); rejected-boundary server HTML contains the Suspense fallback, not the `errorFallback` text (that only appears from the client `Catcher`). Test table updated.
- Confirmed (React 19.3): `use()` on a promise pre-marked `status:'fulfilled'` renders synchronously inside Suspense; an unmarked `Promise.resolve` shows the fallback first. Still undocumented API; hedged.
- Confirmed: `<form action={fn}>` SSRs to `action="javascript:throw new Error('React form unexpectedly submitted.')"` with an event-replay script (queues pre-hydration submits; no no-JS fallback). Documented; also React warns about `method` with a function action.
- Vite 8 claims: `createServer({server:{middlewareMode:true},appType:'custom'})` still the documented route (read current ssr.md); `/@react-refresh` preamble present in plugin-react 6 and in the Vite backend-integration guide; `dist/client/.vite/manifest.json` produced with `--manifest` (built and listed it); entry keyed by source path. Added note that Node cannot run `.tsx` server code (use `tsx` or `ssrLoadModule`).
- `Document` must embed `critical` data (lesson's "rough edge 1"); I implemented that to run the tests.

Project 10: capstone

- **Corrected (wrong code):** the §7.1 router glue returned `children` from a route's `lazy` ; React Router does not allow `lazy` to supply `children` / `path` / `index` , and the module URL 404s (reproduced on 7.18 and 8.5). Replaced with `patchRoutesOnNavigation` + `patch('root', ...)` ; verified on both versions (first visit loads module, nested route renders in the module boundary, unknown URL reaches NotFound).
- Confirmed by execution: `VehicleStore` (per-frame batching, stale-seq drop, stable `getIds` snapshot, resync keeping newer buffered deltas), API client (`AbortSignal.any/timeout` , TimeoutError vs caller abort vs 409/schema mapping), `shouldRetry` , status machine, rate-limited announcer digest, socket backoff/reconnect/ `close()` .
- Confirmed: `onCaughtError/onUncaughtError/onRecoverableError` signatures typecheck on 19.3. Manifest path comment resolved. React Compiler status note (1.0 stable).

Project 08: enterprise modular app

- **Corrected (real defect):** in the ESLint flat-config excerpt the `domain/**` block replaces (not merges) the `no-restricted-imports` options from the earlier `modules/*/**` block, so domain files

silently escaped the deep-import ban (verified with ESLint 10.13 fixtures). Config fixed (R2 repeated; `patterns` must be all strings or all objects, tested).
- Confirmed: glob negations (`!@/modules/*/contracts` , `!.../index`) behave as stated; self-import via alias is flagged (as the lesson warned); `dependency-cruiser` `cruise()` API shape (`await cruise(files,{validate,ruleSet})` , `output.summary.violations[].rule.name` ; `exitCode` stays 0 so assert on violations).
- Confirmed: pricing examples (1999*21% -> 420; 333 -> 70); manifest/ports/registry/domain code typechecks; `usePort` branded typing works (and accepts a raw string key as `unknown` , now noted).

Project 07: component library

- **Corrected (wrong test):** the Tabs keyboard table expected `['ArrowLeft','Profile']` starting from the first tab; ArrowLeft wraps to `Security` . Fixed (ran all four cases).
- **Added caveat (verified):** with `asChild` , a child's own `onClick` runs before Button's loading/disabled guard (consumer handlers composed first), so it is not blocked.
- Confirmed: `composeRefs` mixing legacy fn ref + object ref + cleanup ref works on React 19.3 including StrictMode (attach, detach, attach); unstable `composeRefs` per render re-attaches every render (3 attaches); `useControllableState` flip warning and no-op `onChange` ; `Slot` / `Button` /polymorphic `Box` types (wrong `as` /props fail to compile).
- Library build: Vite 8 emits `dist/index.js` , `dist/primitives/tabs.js` matching the `exports` map; `__dirname` triggers a Vite 8 warning, replaced by `import.meta.dirname` . `useEffectEvent` stable since 19.2 (exported in 19.3), `inert?: boolean` in `@types/react` 19, Compiler 1.0 stable.

Project 06: e-commerce server state

- Confirmed by execution (TanStack Query 5.104): `scope` serializes mutations (max in flight 1, server order 2,3,4, display monotone, final 4); `ensureQueryData` returns stale cache without fetching; `prefetchQuery` fetches stale and skips fresh. All HTTP/query/mutation/checkout code typechecks (incl. `MutationCache.onError` signature, `useActionState`).
- **Corrected:** `useRef(crypto.randomUUID())` calls `randomUUID` on every render (impure render work); replaced with lazy ref init.
- Note: the cart "last mutation wins" strategy is only correct against a server whose GET reflects the last write (documented in the verification note).

Project 05: data-heavy UI

- **Corrected (TS errors under the lesson's own strict config):** `SelectionStore.selectRange` indexes `orderedIds[i]` (`string | undefined`); the worker's `res` / `page` hit TS7022 (circular inference). Both fixed in the text.
- **Corrected:** `maxPages` was configured without `getPreviousPageParam` ; TanStack's infinite-query guide documents `maxPages` together with both, and without it `hasPreviousPage` is false so dropped pages can't be re-fetched upward. Added `getPreviousPageParam` and explanation.
- **Added:** `parseFilters` drops *all* valid siblings when one param is invalid (verified).

- Confirmed: Zod 4 `z.string().date()` still works, filter round trip, virtualized grid renders ~18 rows in jsdom with `clientHeight` stubs, selection range/anchor logic and per-id notifications. Long Tasks API is Chromium-only. `react-dom/profiling` exists in 19.3. React Compiler note updated for Vite 8/plugin-react 6 (`reactCompilerPreset` + `@rolldown/plugin-babel`).
- Unresolved: transition + URL update behavior with the SearchBox effect sync (`useEffect(() => setText(value), [value])` may briefly revert fast typing) was not tested.

Project 04: complex form

- Confirmed: all §8 modules typecheck (strict, noUncheckedIndexedAccess) after supplying the two files the text only names (`ApiContext` , `CheckboxField`); §9.2 reducer tests (8) pass; IBAN vectors (GB82..., DE89..., lowercase/no spaces valid; one-digit mutation, wrong length invalid) and year boundaries behave as stated; out-of-order registration check scenario (latest wins, first signal aborted, one request under StrictMode).
- **Corrected (verified trap):** the lesson told readers to use `vi.useFakeTimers()` + `userEvent.setup({advanceTimers})` ; with Vitest 3 + RTL 16 `user.type` then hangs until timeout (RTL only drains timers for Jest fake timers). Working recipe documented (`shouldAdvanceTime: true`).
- Corrected the StrictMode note (only the debounce timer is cleared/restarted; no request sent twice). Form `action` reset semantics confirmed against react.dev (only uncontrolled fields reset). a11y tooling note (`vitest-axe` is 0.1.0).
- Not run: full wizard integration scenarios 1, 2, 5-10 (BillingStep/ReviewStep are not given in the lesson). **Unresolved.**

Project 02: task manager

- Typechecked and ran all tests (reducer 6, integration 4, render-log 1): 11 pass once the elided `seed.ts` , `Sidebar` , `TaskPanel` , `state/index.ts` are supplied.
- **Corrected:** `getByRole("button", { name: /work/i })` also matches "Delete project Work" (ambiguous); anchored to `/^work$/i` .
- `useEffectEvent` status resolved (stable 19.2). Removed resolved ⚠ markers.

Project 01: interactive fundamentals

- Typechecked all §8 modules; the three §9 tests pass; reintroducing `key={index}` makes the regression test fail as claimed; `useRef` typing claim confirmed.
- Version note updated (Vite 8 needs Node 20.19+/22.12+).

Unresolved / not verified

- P04: BillingStep/ReviewStep-dependent integration scenarios (not in lesson); axe scans; beforeunload behavior (relied on MDN, not run in a browser).
- P05: SearchBox `useEffect` sync under rapid typing; export Worker (type-checked only); CSV-injection guard is described but not implemented in the snippet.
- P06/P08: MSW/Playwright/Storybook/Changesets/size-limit pieces were not built.

- P07: Dialog/Toast/Menu/Select sketches are abridged and were not executed; `exports` map checked by building only (no consumer install, no publint/attw).
- P09: no real browser (hydration was not exercised end to end; gzip `Z_SYNC_FLUSH` and compression/proxy buffering, CSP enforcement, Playwright JS-off journeys not run).
- P10: only the code shown was run; MSW handlers, i18n, CI/ESLint config excerpts and the perf-budget script were not executed.
- Bundle-size numbers in P11 are from my build and are hardware/version dependent (the lesson already says illustrative).